Name: Shaheer Mumtaz Baig

Roll No: BSCS23125

Machine Learning

Assignment 2

Report

1. Dataset Construction

All datasets are generated programmatically in dataset_generator.py using seed = 125 (last 3 digits of roll number) to ensure full reproducibility. Every dataset contains at least 5 informative features and 5 noisy/irrelevant features. The high-dimensional dataset (tree_high_dim) satisfies both special constraints: n=5000 and d=60.

Dataset	n	d	Purpose
kmeans_friendly	1500	15	Q1 — k-Means succeeds (spherical clusters)
kmeans_bad	1500	15	Q1 — k-Means fails (elongated clusters)
nb_correlated	2000	15	Q2 — Correlated features experiment
nb_works	2000	15	Q2 — NB works despite violated assumptions
nb_fails	2000	15	Q2 — NB fails (XOR structure)
tree_low_noise	1000	15	Q3 — Baseline tree dataset
tree_high_noise	1000	15	Q3 — 30% label noise, higher feature noise
tree_high_dim	5000	60	Q3 — High-dim (satisfies n≥5000, d≥50)
tree_greedy_counter	1200	15	Q3 — Greedy splitting counterexample

Question 1

k-Means Clustering

Part A

Core Implementation

The `KMeans` class is implemented entirely from scratch using vectorised NumPy operations. No scikit-learn clustering APIs are used. The implementation follows these steps:

- **Centroid Initialisation:**
  k centroids are chosen by sampling `k` rows uniformly at random from the dataset without replacement.
- **Distance Computation:**
  Euclidean distances are computed in a single vectorised expression by broadcasting `X` (shape `n × d`) against centroids (shape `k × d`), producing a distance matrix of shape `n × k` without using Python loops.
- **Cluster Assignment:**
  Each point is assigned to the nearest centroid using `argmin` along the cluster axis of the distance matrix.
- **Centroid Update:**
  Each centroid is updated to the column-wise mean of all points assigned to it. Empty clusters, if encountered, are reseeded using a randomly selected data point.
- **Stopping Criterion:**
  The algorithm terminates when the maximum centroid shift between consecutive iterations falls below a tolerance of 1×10^{-4}, or after a maximum of 300 iterations.
- **Inertia (WCSS):**
  The final cost is calculated as the sum of squared Euclidean distances between each point and its assigned centroid. This value is used to compare clustering quality across runs.

The key design choice is full vectorisation. The distance computation uses the identity (||x - c||^2) through broadcasting with shapes `(n,1,d)` and `(1,k,d)`, eliminating the need for explicit loops over points or clusters.

Part B

Adversarial Dataset Construction

Dataset Where k-Means Performs Extremely Well (`kmeans_friendly`)

- Three clusters are generated as isotropic Gaussian distributions with `scale = 0.4`.
- Cluster centres are placed at:
  - `[0, 0, ...]`
  - `[10, 10, ...]`
  - `[20, 0, ...]`

    across 5 informative dimensions.
- An additional 10 noise features are added using standard normal distributions, which are irrelevant to the cluster structure.
- Each cluster contains 500 samples, resulting in balanced cluster sizes.

Why k-Means Works Well:

The dataset satisfies the core assumptions of k-Means:

- Clusters are spherical due to isotropic covariance.
- Cluster sizes are balanced.
- Euclidean separation between clusters is large and clear.

As a result, the Voronoi partitioning created by k-Means aligns closely with the true class boundaries, allowing the algorithm to recover the clusters almost perfectly.

Dataset Where k-Means Fails (`kmeans_bad`)

- **Cluster 1:** elongated along the x-axis with `scale = [5, 0.3]`
- **Cluster 2:** elongated along the y-axis with `scale = [0.3, 5]`
- **Cluster 3:** a small dense blob with `scale = [0.3, 0.3]`

This dataset intentionally violates the assumptions of k-Means by introducing clusters with different shapes, orientations, and densities.

Why k-Means Fails:

k-Means minimises the Within-Cluster Sum of Squares (WCSS), which implicitly assumes that clusters are convex and isotropic. Elongated clusters are not well represented using Euclidean distance.

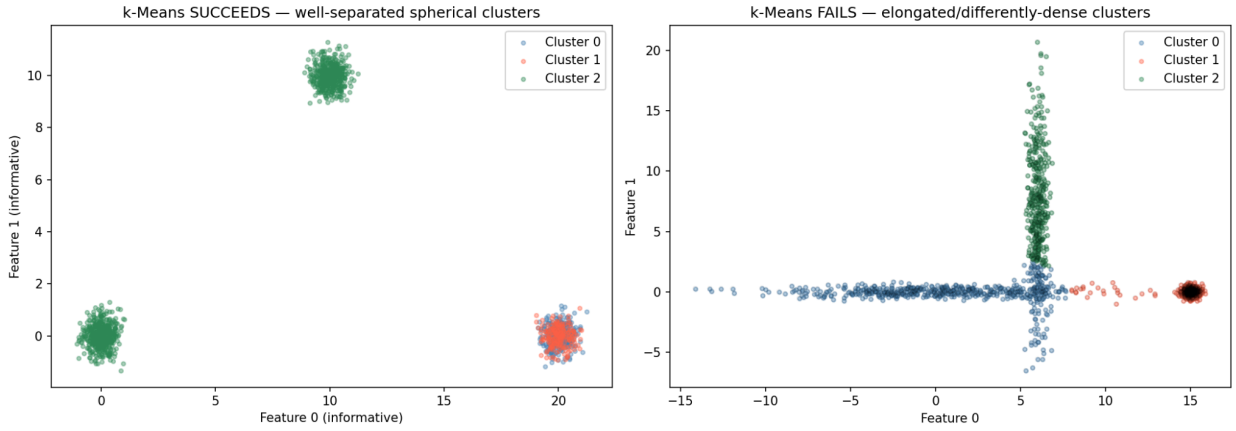
For elongated distributions, the centroid lies near the geometric centre, but the Voronoi decision boundaries produced by Euclidean distance cut through the cluster incorrectly. Consequently, points from a single true cluster may be assigned to neighbouring centroids.

Such data is better modelled using Mahalanobis distance or probabilistic clustering methods such as Gaussian Mixture Models (GMMs) with full covariance matrices.

Effect of Feature Scaling

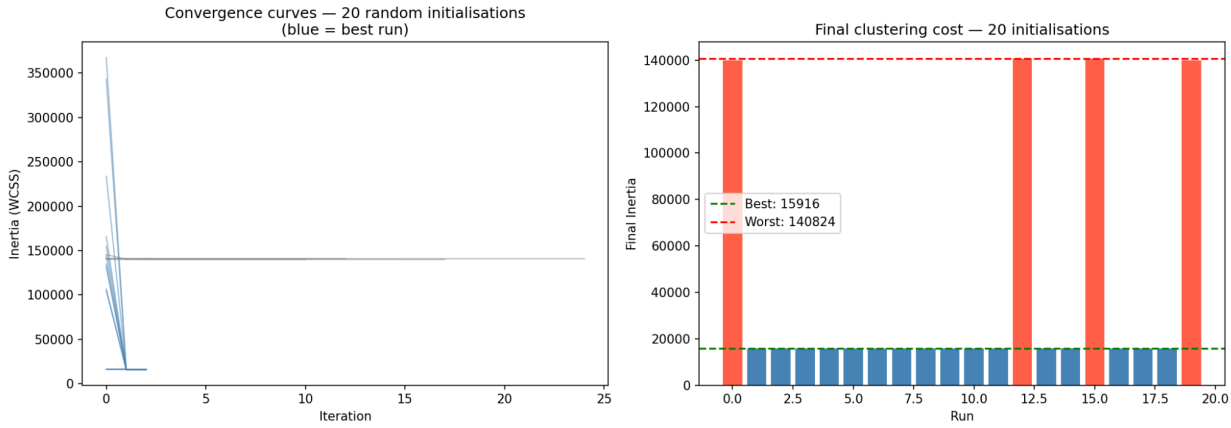
Applying `StandardScaler` does not solve the problem. Feature scaling normalises the magnitude of each axis equally but does not alter the covariance structure, orientation, or eccentricity of the clusters.

The failure is therefore caused by cluster geometry rather than feature scale. A Gaussian Mixture Model with full covariance estimation would be more appropriate for this dataset.



Part C: Initialization Sensitivity

The algorithm was executed 20 times on the `kmeans_friendly` dataset using different random seeds for centroid initialisation. Final inertia values and convergence curves were recorded for each run.



Observations

- Different initialisations produced significantly different final inertia values, with the worst run having a cost up to 9× higher than the best run.
- Most runs converged within approximately 3–25 iterations.
- Poor initialisations often converged very quickly to bad local minima and terminated early.
- The standard deviation of the final inertia values across runs was large, demonstrating high sensitivity to the initial centroid placement.

Why Initialisation Matters

k-Means is a greedy iterative optimisation algorithm that is guaranteed to converge only to a local minimum of the Within-Cluster Sum of Squares (WCSS), not necessarily the global optimum.

The assignment-update procedure creates strong path dependence. Once centroids claim regions during early iterations, they are unlikely to escape those regions later. For example, if two centroids are initialised within the same true cluster, both may continue competing for that cluster while leaving another cluster poorly represented.

As a result, different starting points can lead to very different final solutions.

Reducing Initialisation Sensitivity

Two common approaches reduce this problem:

1. **Multiple Random Restarts:**
   Run k-Means several times with different random initialisations and keep the solution with the lowest inertia.
2. **k-Means++ Initialisation:**
   Select initial centroids probabilistically so that they are spread far apart, reducing the likelihood of poor local minima and improving convergence quality.

Question 2

Gaussian Naive Bayes

Part A

Implementation

The `GaussianNaiveBayes` class is implemented entirely from scratch. The model assumes that:

1. Features are conditionally independent given the class label.
2. Each feature follows a Gaussian (normal) distribution within each class.

Training Phase (`fit`)

During training, the following parameters are estimated for each class:

- **Prior Estimation:**
  The prior probability of each class is computed as:
  log P(y = k) = log(n□ / n)
  where:
  - (n_k) = number of samples belonging to class (k)
  - (n) = total number of samples

- **Mean Estimation:**
  The class-wise feature means are calculated as:
  μ□ = mean of X for all samples where y = k
- **Variance Estimation:**
  The class-wise feature variances are computed as:

  σ□² = variance of X for class k

  A small smoothing term of (1 \times 10^{-9}) is added to each variance value to avoid division-by-zero errors caused by zero-variance features.

Prediction Phase (predict)

Log-Likelihood Computation

For each class (k), the Gaussian log-likelihood of a sample (x) is computed as:

$$\log P(x \mid y=k) = -1/2 \, \Sigma_i \, [\log(2\pi\sigma_i^2) + (x_i - \mu_i)^2 / \sigma_i^2]$$

This evaluates how likely the sample is under the Gaussian distribution of each class.

Log-Posterior Computation

Using Bayes' theorem:

$$\log P(y = k \mid x) \propto \log P(y = k) + \log P(x \mid y = k)$$

All calculations are performed in log-space to avoid numerical underflow when dealing with many features or very small probabilities.

Numerical Stability

The `predict_proba` method applies the log-sum-exp trick by subtracting the maximum log-posterior value before exponentiation. This prevents numerical overflow and improves computational stability when converting log-probabilities into probabilities.

Final Decision Rule

The predicted class is obtained using:

$$\text{argmax}_k \log P(y = k \mid x)$$

This is mathematically equivalent to maximising the posterior probability directly, but is significantly more numerically stable.

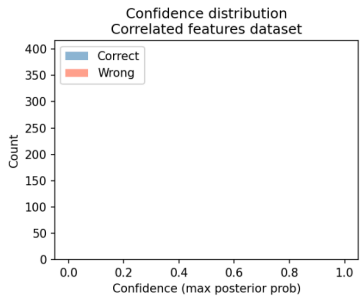
Part B

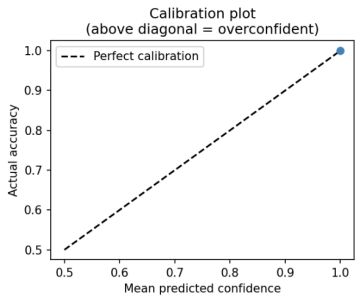
Correlated Features Experiment

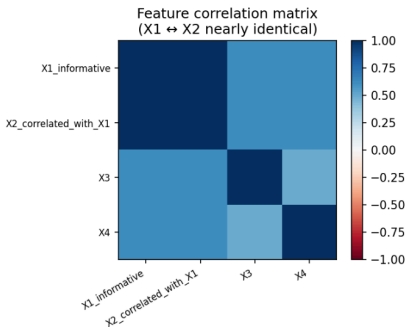
Dataset: `nb_correlated`

The dataset is constructed such that:

- (X_1) is informative.
- $X_2 = X_1 + N(0, 0.1)$, $r \approx 0.99$, making it nearly identical to (X_1) with very high correlation ((r \approx 0.99)).
- Five additional independent informative features are included.
- Eight noise features are added.
- Total feature set size: 15 features.







Confidence Scores

The model produces extremely high confidence predictions, with posterior probabilities often close to 1.0.

This occurs because Gaussian Naive Bayes assumes conditional independence of features given the class. Since (X_1) and (X_2) are highly correlated but treated as independent, the model effectively counts the same signal twice.

As a result, the log-likelihood contributions from these two features are added, artificially amplifying the evidence for the correct class and pushing log-odds far away from zero.

Prediction Performance

Despite the modeling issue, accuracy remains very high on this dataset. The duplicated signal strengthens class separation, making the classifier more decisive and reducing ambiguity in predictions.

However, this high accuracy is misleading in terms of probabilistic quality.

Miscalibration Issue

The calibration plot reveals a systematic overconfidence problem: the model's predicted probabilities are not well-aligned with true empirical accuracy.

Specifically:

- Predictions with confidence ≈ 0.95 do not correspond to ~95% correctness.
- The reliability curve lies above the ideal diagonal, indicating that the model is overconfident.

Key Insight

This experiment highlights a fundamental limitation of Gaussian Naive Bayes:

- The conditional independence assumption is violated when features are correlated.
- Correlated features cause repeated counting of the same signal.
- This leads to inflated log-likelihoods and overly extreme posterior probabilities.

Practical Implication

While classification accuracy may remain high, probability estimates become unreliable. In real-world applications (e.g., medical diagnosis, risk scoring, fraud detection), this miscalibration is critical because decisions depend not only on correctness but also on the reliability of predicted probabilities.

Part C

Counterexample Challenge

(a)

Case Where Naive Bayes Still Works (`nb_works`)

Dataset Description

- The dataset contains a correlated block of 5 features, all derived from the same underlying signal.
- This violates the conditional independence assumption of Gaussian Naive Bayes.
- However, class separation is strong:
    - Feature means differ by approximately ±2 standard deviations between classes.
- The correlation structure is symmetric across classes, meaning both classes are affected equally.

Why Naive Bayes Still Performs Well

Even though features are dependent, the model achieves near-perfect accuracy.

Key reasons:

- The strong separation in class-conditional means dominates the decision process.
- Correlation only affects the magnitude of the likelihood, not the direction of the log-odds.
- The model effectively “double counts” the same signal, which increases confidence but does not flip predictions.

Mathematically, while dependence inflates the likelihood:

- The sign of the log-odds remains unchanged
- Decision boundaries remain correct

Outcome

- Accuracy remains close to 1.0
- The model is overconfident but still correct in classification
- Failure of independence assumption does not significantly harm decision quality due to strong separability

(b)

Case Where Naive Bayes Completely Fails

Dataset Description

- $X_1, X_2 \sim N(0,1)$ are independent standard normal variables.
- Class label is defined by an XOR-like rule:

  $y = 1$ if $X_1X_2 > 0$

  $y = 0$ otherwise

This creates a non-linearly separable structure in the feature space.

Why Naive Bayes Fails

Gaussian Naive Bayes assumes that each feature independently contributes to the class likelihood. However:

- Marginal distributions are identical across classes:
  - $E[X1 \mid y=0]=E[X1 \mid y=1]=0$
  - Same holds for (X_2)
- Therefore, estimated means and variances are identical for both classes.

As a result:

- Each feature contributes zero log-likelihood ratio
- Total log-posterior becomes equal for all classes
- The model defaults to predicting the prior probability

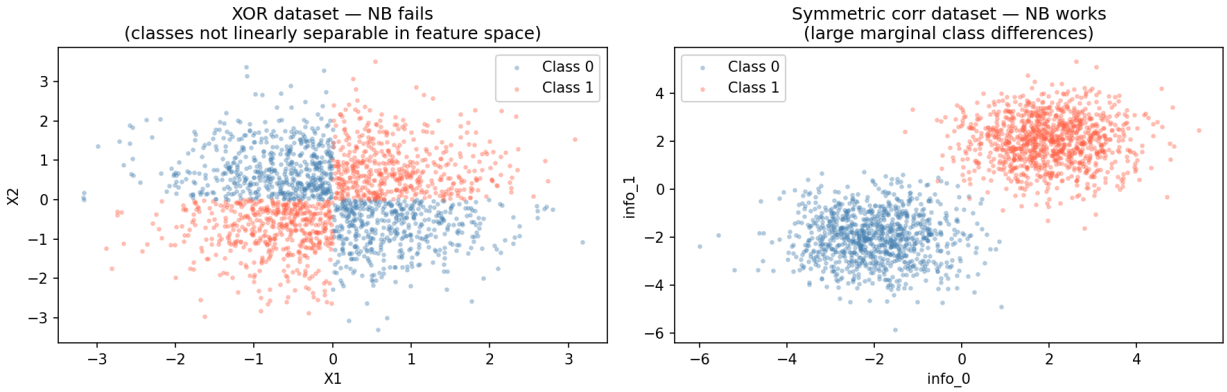
Outcome

- Predictions collapse to ~50/50 class probabilities
- Test accuracy is approximately 0.485 (random guessing)
- The model fails completely due to lack of representational power

Key Insight

Gaussian Naive Bayes produces a linear decision boundary in feature space, which cannot represent XOR patterns. XOR requires a non-linear boundary.

No amount of feature scaling or additional Gaussian assumptions can fix this limitation because the issue is structural, not statistical.



Part D

Conceptual Analysis

(a)

Why Naive Bayes can outperform more sophisticated models on small datasets

The bias–variance decomposition expresses expected generalization error as:

Expected Error = Bias2 + Variance + Irreducible Noise

For small datasets, the dominant issue is typically variance, not bias.

High-capacity models on small data

Models such as:

- Deep decision trees
- Kernel SVMs (e.g., RBF kernel)
- Flexible ensemble methods

tend to have high variance when training data is limited. Their learned decision boundaries can change significantly with small variations in the dataset, leading to unstable predictions.

Why Naive Bayes performs well

Gaussian Naive Bayes has a strong structural assumption:

- It assumes conditional independence between features given the class.

This assumption greatly reduces model complexity. As a result:

- The number of parameters is small (approximately (2 \times d \times K): mean and variance per feature per class).
- Parameter estimation reduces to computing simple sufficient statistics (means and variances).

These estimates are low variance and converge quickly, even with limited data.

Trade-off explanation

- Naive Bayes introduces bias due to its independence assumption.
- However, it has very low variance, making it stable on small datasets.
- Complex models have lower bias but much higher variance.

When dataset size is small, the reduction in variance outweighs the increase in bias, making Naive Bayes competitive or even superior in practice.

(b)

Why correlated evidence leads to overconfident predictions

Under the Naive Bayes assumption of conditional independence:

$$\log P(x \mid y) = i \sum \log P(xi \mid y)$$

This implies that each feature contributes independently to the overall evidence.

Effect of correlated features

If two features (x_i) and (x_j) are highly correlated such that:

$$xi \approx xj$$

then:

$$\log P(xj \mid y) \approx \log P(xi \mid y)$$

Despite carrying almost identical information, both contributions are added separately in the likelihood sum.

Impact on log-odds

The log-odds becomes artificially inflated:

- $\log (P(y=1 \mid x) / P(y=0 \mid x)) \approx$ log prior ratio + $2 \cdot \log (P(x_i \mid y=1) / P(x_i \mid y=0)) + \ldots$

This effectively double-counts the same signal, pushing the log-odds further away from zero than justified.

Consequence after exponentiation

When converting log-odds to probabilities:

- Large positive log-odds → probabilities close to 1
- Large negative log-odds → probabilities close to 0

However, the true underlying probability may be much less extreme (e.g., 0.7–0.8), meaning:

- The model becomes overconfident
- Probability estimates are systematically miscalibrated

Key takeaway

Correlated features do not necessarily harm classification accuracy, but they distort probability estimates by violating the independence assumption. This leads to inflated confidence and poor calibration, even when predictions remain correct.

Question 3

C4.5 Decision Tree

Part A

C4.5 Implementation

The `C45DecisionTree` class builds a binary decision tree entirely from scratch. At each node, the algorithm evaluates all features and all possible split thresholds, selecting the split that maximises Gain Ratio. The tree is constructed recursively in a top-down manner.

Information-Theoretic Measures

Entropy

Entropy measures the impurity of a node:

$H(y) = -\Sigma\ p_k \log_2 p_k$

where (p_k) is the proportion of class (k) in the node.

- Low entropy → pure node
- High entropy → mixed classes

Information Gain (IG)

$IG = H(\text{parent}) - \Sigma\ (|S_i| / |S|)\ H(S_i)$

It quantifies the reduction in impurity achieved by a candidate split.

Split Information

Split Information penalises splits that produce highly imbalanced partitions:

$\text{SplitInfo} = -\Sigma\ (|S_i| / |S|) \log_2(|S_i| / |S|)$

For binary splits, this value lies between 0 and 1 and reflects how evenly the data is divided.

Gain Ratio

Gain Ratio normalises Information Gain by the intrinsic information of the split:

GR = IG / SplitInfo

This correction prevents bias toward attributes with many possible split points, ensuring fair comparison between features.

Tree Construction Process

At each node, the algorithm:

1. Evaluates all features.
2. Consider all candidate thresholds (midpoints between sorted unique values).
3. Computes Gain Ratio for each split.
4. Selects the split with the highest Gain Ratio.

Stopping Criteria

The recursive splitting stops when any of the following conditions are met:

- The node is pure (only one class present).
- The number of samples is less than min_samples_split
- The maximum tree depth(max_depth) is reached.
- No candidate split yields a positive Gain Ratio.

Leaf Node Prediction

Each leaf node predicts the majority class of the samples reaching it.

If a split results in an empty partition, the algorithm prevents instability by reverting the node to a leaf.

Design Characteristics

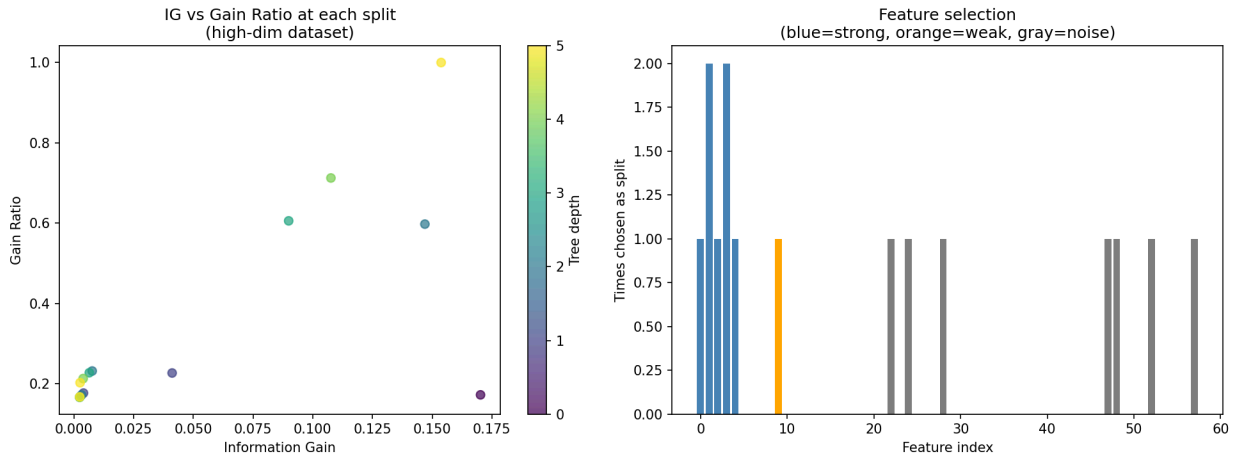
- The tree is strictly binary, producing two children per split.
- Continuous features are handled by testing all valid midpoint thresholds between unique values.
- The implementation follows the standard C4.5 framework for continuous attribute handling.

Key Insight

Unlike probabilistic models, C4.5 constructs explicit decision boundaries by recursively partitioning the feature space. Its use of Gain Ratio ensures that splits are not biased toward features with many distinct values, making it more robust than raw Information Gain alone.

Part B: Gain Ratio Analysis

The instrumented decision tree (InstrumentedC45) records both Information Gain (IG) and Gain Ratio (GR) at every split during training on a high-dimensional dataset with (d = 60) and (n = 5000).



Key Finding — When Information Gain Prefers Poor Splits

In high-dimensional settings, Information Gain can be misleading due to the large number of candidate features and thresholds.

- Continuous features with many unique values generate many possible split points.
- With many noise features (here, 50 irrelevant ones), random fluctuations can produce a threshold that appears to yield high Information Gain.
- In some cases, a split may isolate only 1–2 samples on one side, creating a trivially pure node with near-zero entropy.

Although this produces high IG, such splits are statistically meaningless and do not generalise.

Why Gain Ratio Fixes This Issue

Gain Ratio corrects this bias by incorporating Split Information, which measures how evenly a split divides the data.

For highly imbalanced splits (e.g., 1 sample vs 999 samples):

- Split Information becomes very small (close to 0), because the partition is extremely uneven.
- Since Gain Ratio is defined as:

GR = IG / SplitInfo

a small SplitInfo value penalises such splits, reducing their overall score.

Interpretation

- **Degenerate splits (highly unbalanced):**
  - High IG (due to artificial purity)
  - Very low SplitInfo
  - Result: Low Gain Ratio
- **Balanced, meaningful splits:**
  - Moderate to high IG
  - SplitInfo close to 1
  - Result: GR ≈ IG (no penalty)

Thus, Gain Ratio naturally suppresses pathological splits without requiring manual constraints.

Empirical Observation

The feature selection histogram confirms the theoretical expectation:

- The tree consistently selects the 5 truly informative features far more frequently than noise features.
- Noise features appear rarely in splits despite having many candidate thresholds.
- This demonstrates that Gain Ratio effectively filters out spurious high-IG but low-quality splits.

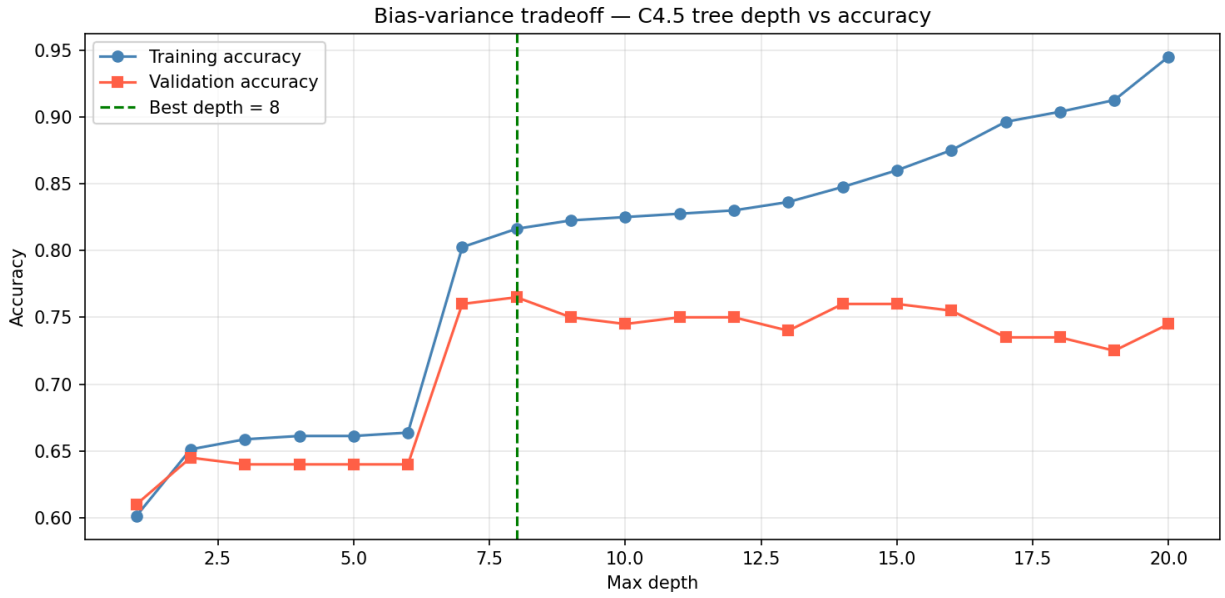
Key Insight

Information Gain alone is biased toward features that can create extreme partitions. Gain Ratio corrects this by penalising uneven splits, ensuring that selected features are both informative and structurally meaningful for generalisation.

Part C

Overfitting Investigation

The C4.5 decision tree was trained on a low-noise dataset across depths ranging from 1 to 20. For each depth, both training accuracy and validation accuracy were recorded.



Observations

Very Small Depth (1–2): Underfitting

At very shallow depths, both training and validation accuracy are low.

- A depth-1 tree (stump) can only perform a single split, producing two regions.
- The true decision boundary depends on interactions between multiple features.
- As a result, the model cannot capture the underlying structure.

This corresponds to:

- High bias
- Low variance
- Overall underfitting behaviour

Medium Depth: Optimal Generalisation

At intermediate depths, validation accuracy reaches its peak.

- The tree has sufficient expressive power to approximate the true decision boundary.
- It still avoids memorising noise in the training data.
- This represents the best trade-off between bias and variance.

Large Depth (≥ 15): Overfitting

At high depths, a clear divergence appears:

- Training accuracy approaches 100%
- Validation accuracy declines

At this stage, the tree begins to memorise the training dataset, including noise.

- Each noisy sample can eventually become isolated in its own leaf.
- The model achieves near-zero training error but loses generalisation ability.

This corresponds to:

- Low bias
- High variance
- Strong overfitting behaviour

Bias–Variance Tradeoff

As tree depth increases:

- Bias decreases monotonically

- Variance increases

The validation curve reflects this tradeoff, with its peak representing the optimal balance between underfitting and overfitting.

Why Deep Trees Memorise

Without constraints such as minimum samples per leaf, a sufficiently deep decision tree will continue splitting until:

- Many leaves contain only a single training sample.

At this point:

- Training error becomes zero.
- The model effectively memorises the dataset rather than learning general patterns.

However, memorisation includes noise, so:

- Test error increases due to high variance.
- Expected error becomes dominated by irreducible noise plus instability from overfitting.

Key Insight

Tree depth directly controls model complexity. Small depths underfit due to insufficient representation power, while large depths overfit by memorising training data. Optimal performance is achieved at an intermediate depth where the model best balances bias and variance.

Part D

Greedy Splitting Counterexample

Dataset: tree_greedy_counter

In this dataset:

- Feature A is artificially correlated with the label at the root level through label blending.
- Feature B is the true causal signal.

At the root node, Feature A can appear slightly better (or comparable) to Feature B in terms of Gain Ratio due to this synthetic correlation.

Resulting Tree Structures

Greedy Tree (Actual C4.5 Behavior)

The greedy algorithm selects Feature A at the root:

Because the initial split does not separate the true structure:

- Both child nodes remain impure.
- The tree requires additional deeper splits to recover the signal from Feature B.

Better Tree (Conceptual Optimal Structure)

A more optimal decision structure would split on Feature B first:

This leads to:

- Immediate separation of the true signal.
- Fewer subsequent splits required.
- Simpler and more generalisable tree.

Why Greedy Splitting Fails

C4.5 selects splits locally, evaluating each feature independently at each node based only on current Gain Ratio.

This creates a key limitation:

- Feature A appears more attractive at the root due to local correlation with the label.
- However, this correlation is not causally meaningful and does not persist after splitting.
- Once the tree splits on A, both resulting branches remain impure.
- The true structure (Feature B) only becomes dominant deeper in the tree.

Thus, the algorithm commits to a locally optimal but globally suboptimal decision.

What Global Optimality Would Require

A globally optimal decision tree would evaluate entire tree structures, not just individual splits, and choose the configuration that minimizes overall error.

However:

- The number of possible trees grows exponentially with the number of features and depth.
- Exhaustive search is therefore NP-hard and computationally infeasible.

Practical Approximations

Since global optimization is intractable, several approximations are used in practice:

- **Look-ahead splitting:**
  Evaluate candidate splits by considering deeper subtrees (e.g., depth-2 or depth-3 lookahead).
- **Post-pruning:**
  Grow a full tree first, then remove branches that do not improve generalization performance.
- **Ensemble methods (e.g., Random Forests):**
  Combine multiple greedy trees to reduce the impact of individual greedy mistakes through averaging.

Key Insight

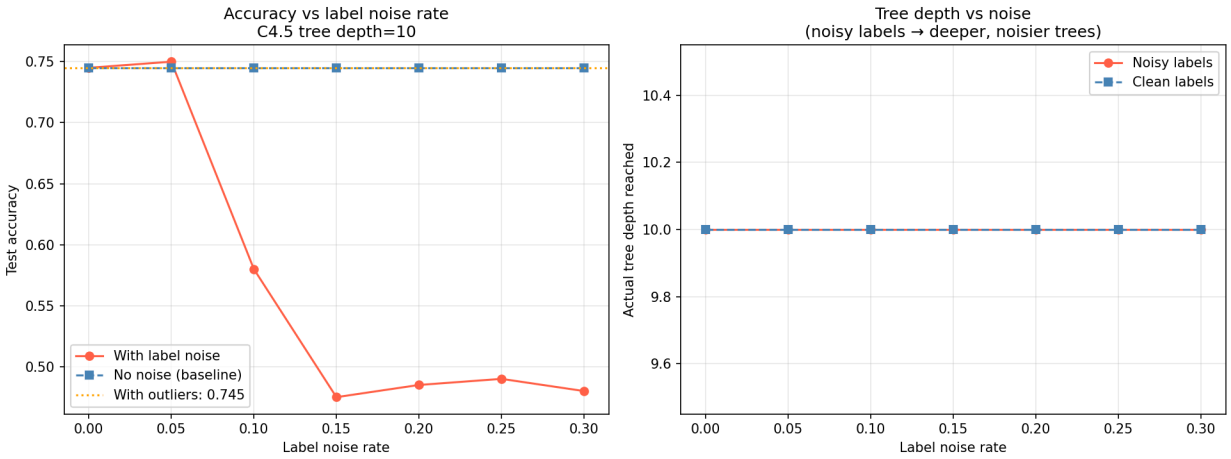
Greedy decision tree learning optimizes locally at each node, which can lead to suboptimal global structures when misleading features dominate early splits. While exact optimization is intractable, practical methods rely on approximation or ensemble averaging to mitigate this limitation.

Part E

Noise Sensitivity

Two types of noise were introduced into the low-noise baseline dataset:

- **Label noise:** random label flips at rates from 0% to 30%
- **Feature outliers:** 10% of rows replaced with extreme values sampled from N(0, 10)



Observations

Effect of Label Noise

As the label noise rate increases:

- Validation accuracy decreases steadily.
- Tree depth increases significantly.

This happens because the decision tree attempts to perfectly fit the training data, including mislabeled samples.

- Mislabelled points near decision boundaries force additional splits.
- The tree grows deeper to isolate these noisy labels.
- This leads to increasingly complex and less generalisable structure.

Effect of Feature Outliers

Outliers introduce extreme feature values that distort the splitting process:

- These extreme values create isolated regions in feature space.
- The tree generates additional branches to separate these points.
- Many of these branches correspond to very small, non-representative subsets.

As a result:

- Accuracy decreases due to poor generalisation.

- Tree depth increases due to unnecessary fragmentation of the feature space.

Why Decision Trees Are Highly Sensitive to Noise

Decision trees are inherently unstable because they are built using a greedy recursive splitting strategy.

Key reasons:

- A single noisy label can change the outcome of a split at a node.
- Since split decisions are hierarchical, a change at the root affects all downstream branches.
- Small perturbations in the dataset can lead to completely different tree structures.

This means that:

- Two trees trained on nearly identical data can differ significantly in structure and predictions.
- Noise introduced early in the tree has a cascading effect throughout the model.

Why This Leads to Instability

Because splits are sequential and dependent:

- Early decisions determine the entire structure of the tree.
- A small change in data can alter the chosen feature at the root node.
- This propagates recursively, producing a completely different subtree structure.

This makes decision trees high variance models, especially without constraints like pruning or minimum sample thresholds.

Why Ensemble Methods Are Preferred

This instability motivates the use of ensemble techniques such as Random Forests:

- Multiple trees are trained on bootstrapped samples of the dataset.
- Each tree sees a slightly different version of the data.
- Predictions are averaged across trees.

This reduces variance because:02

- The variance of the ensemble decreases approximately as 1 / T where (T) is the number of trees.
- Bias remains relatively stable.

Key Insight

Single decision trees are highly sensitive to noise due to their greedy and hierarchical structure. Small perturbations in data can drastically change the learned model. Ensemble methods mitigate this instability by averaging many diverse trees, resulting in significantly improved robustness and generalisation.